

Méthodes Formelles de Développement - Correction DS 23-24

Exercice 1

1.- Calculer les substitutions suivantes et montrer qu'elles donnent True.

-/ [ANY xx WHERE xx:NAT & xx>5 THEN yy:=xx END] yy>5

[ANY x WHERE P THEN S END] I $\equiv \forall x.(P \Rightarrow [S]I)$

[ANY xx WHERE xx:NAT & xx>5 THEN yy:=xx END] (yy > 5)
 $\equiv \forall xx.(xx:NAT \wedge xx>5) \Rightarrow [yy:=xx] (yy > 5)$
 $\equiv \forall xx.(xx:NAT \wedge xx>5) \Rightarrow (xx > 5)$
 $\equiv \forall xx.(TRUE)$ (car la conclusion est incluse dans l'hypothèse; l'implication est une tautologie)
 $\equiv TRUE$

-/ [ANY xx WHERE xx:NAT & xx>0 THEN yy:=xx+1 END] yy>1
 $\equiv \forall xx.(xx:NAT \wedge xx>0) \Rightarrow [yy:=xx+1] (yy > 1)$
 $\equiv \forall xx.(xx:NAT \wedge xx>0) \Rightarrow (xx+1 > 1)$
 $\equiv \forall xx.(xx:NAT \wedge xx>0) \Rightarrow (xx > 0)$
 $\equiv \forall xx.(TRUE)$ (tautologie)
 $\equiv TRUE$

-/ [ANY xx WHERE xx:NAT & xx>yy THEN zz:=xx END] zz>yy
 $\equiv \forall xx.(xx:NAT \wedge xx>yy) \Rightarrow [zz:=xx] (zz > yy)$
 $\equiv \forall xx.(xx:NAT \wedge xx>yy) \Rightarrow (xx > yy)$
 $\equiv \forall xx.(TRUE)$ (tautologie)
 $\equiv TRUE$

-/ [CHOICE xx:=1 OR xx:=2 END] xx>0

[CHOICE S OR T END] I $\equiv [S]I \wedge [T]I$

[CHOICE xx:=1 OR xx:=2 END] (xx > 0)
 $\equiv [xx:=1] (xx > 0) \wedge [xx:=2] (xx > 0)$
 $\equiv (1 > 0) \wedge (2 > 0)$
 $\equiv TRUE \wedge TRUE$
 $\equiv TRUE$

2.- Effectuer les substitutions suivantes :

-/ [xx:=1]($\forall xx.(xx:N \Rightarrow xx>=yy) \vee xx>=0$)

$\equiv ([xx:=1] \forall xx.(xx:N \Rightarrow xx>=yy)) \vee ([xx:=1] (xx>=0))$

La substitution [xx:=1] ne modifie pas les occurrences de xx liées par le quantificateur

$\equiv \forall xx.(xx:N \Rightarrow xx>=yy) \vee 1 >= 0$

$\equiv (\forall xx.(xx:N \Rightarrow xx>=yy)) \vee TRUE$

$\equiv TRUE$

-/ [xx:=0] (yy>0)

$\equiv$ yy>0

Car xx n'apparaît pas dans le prédicat; la substitution n'a aucun effet.

-/ [PRE xx:N THEN yy:=xx END] ($\forall xx.(xx:N \Rightarrow yy>zz)$)

[PRE P THEN S END] I $\equiv$ P $\wedge$ [S]I

[PRE xx:N THEN yy:=xx END] ($\forall xx.(xx:N \Rightarrow yy>zz)$)

$\equiv (xx:N) \wedge [yy:=xx] (\forall xx.(xx:N \Rightarrow yy>zz))$

Il faut effectuer un changement de variable dans ce cas

xx apparaît comme paramètre de la précondition P et la substitution S et comme variable liée au quantificateur dans le I avec $\forall xx$. Pour éviter tout conflit et effectuer les substitutions correctement, il faut renommer xx dans I par aa par exemple:

$\equiv (xx:N) \wedge [yy:=xx] (\forall aa.(aa:N \Rightarrow yy>zz))$

$\equiv (xx:N) \wedge (\forall aa.(aa:N \Rightarrow xx>zz))$

La 2eme partie veut dire que pour tout entier naturel, $xx>zz$. Or $xx>zz$ ne dépend pas de aa;
donc: $(\forall aa.(aa:N \Rightarrow xx>zz)) \equiv xx>zz$

$\equiv (xx:N) \wedge (xx>zz)$

Exercice2

NB: pour sauvegarder la valeur d'une variable v avant l'exécution d'une substitution, on peut utiliser v\$0 qui fait partie de eventB (et non du B tandard), ou utiliser comme ici pour se conformer au B standard
LET unevar BE unevar=v IN ... END

MACHINE Port

SETS

NAVIRES

CONSTANTS

km,

NAVIRES_Personnes,

NAVIRES_Marchandise

PROPERTIES

km : **NAT1** $\wedge$

NAVIRES_Personnes $\subseteq$ NAVIRES $\wedge$

NAVIRES_Marchandise $\subseteq$ NAVIRES $\wedge$

NAVIRES_Personnes $\cup$ NAVIRES_Marchandise = NAVIRES $\wedge$

NAVIRES_Personnes $\cap$ NAVIRES_Marchandise = {}

DEFINITIONS

Quais==1..km

VARIABLES

naviresPresents, naviresEnAttente,

Sur_quai,

tete, queue, suiv

INVARIANT

naviresPresents $\subseteq$ NAVIRES $\wedge$

naviresEnAttente $\subseteq$ naviresPresents $\wedge$

Sur_quai $\in$ (naviresPresents - naviresEnAttente) $\rightarrow$ Quais $\wedge$

naviresEnAttente $\cap$ dom(Sur_quai) = $\emptyset$ $\wedge$

tete $\in$ naviresEnAttente $\cup$ {0} $\wedge$

queue $\in$ naviresEnAttente $\cup$ {0} $\wedge$

```

    suiv ∈ naviresEnAttente → naviresEnAttente ∪ {0} ∧
    ((tete=0) ⇔ (naviresEnAttente={})) ∧
    (tete ≠ 0 ⇒ suiv(tete) ≠ tete) ∧
    (queue ≠ 0 ⇒ suiv(queue)=0) ∧
    ∀nn.(nn ∈ naviresEnAttente ∧ nn ≠ queue ⇒ suiv(nn) ∈ naviresEnAttente ∧
    suiv(nn) ≠ 0)
INITIALISATION
    naviresPresents := {} ||
    naviresEnAttente := {} ||
    Sur_quai := {} ||
    tete := 0 ||
    queue := 0 ||
    suiv := {}
OPERATIONS
    arrive(nav)=
    PRE
        nav ∈ NAVIRES ∧ nav ∉ naviresPresents
    THEN
        naviresEnAttente := naviresEnAttente ∪ {nav} ||
        naviresPresents := naviresPresents ∪ {nav} ||
        IF (tete = 0) THEN
            tete:=nav ||
            queue:=nav ||
            suiv := suiv ∪ {nav ↦ 0}
        ELSE
            suiv:= (suiv <+ {queue ↦ nav}) ∪ {nav ↦ 0} ||
            queue := nav
        END
    END;

    qq ← quitte(nn)=
    PRE
        nn ∈ dom(Sur_quai)
    THEN
        qq:= Sur_quai(nn) ||
        Sur_quai := {nn} ⧹ Sur_quai ||
        naviresPresents := naviresPresents - {nn}
    END;

    mettreAquai (qq) =
    PRE
        qq ∈ Quais ∧ tete ≠ 0 ∧ qq ∉ ran(Sur_quai)
    THEN
        LET atete BE atete=tete IN
            Sur_quai := Sur_quai ∪ {atete ↦ qq} ||
            tete:=suiv(atete) ||
            suiv:= {atete} ⧹ suiv ||
            naviresEnAttente := naviresEnAttente - {atete} ||
            IF tete=0
            THEN
                queue:=0
            END
        END
    END;

    END;

    nn ← navireAttente =
    PRE
        tete ≠ 0
    THEN
        nn:=tete
    END
END

```